

抖音脚本：用 Rust 写了一个 AI 写作 Agent

时长：约 7-9 分钟 | 风格：坐镜头前直接说，偶尔切屏幕录制

开场（45 秒） — 先看成品

（屏幕录制：终端里 `cargo run` → REPL 提示符 → 打字"写一篇关于认知偏差在投资中的应用，轻辩风格" → Agent 自动开始 spinner，十几秒后展示全文）

这个项目叫 cognitive-writer，基于 Rust，整条链路跑在终端里。

它的工作是：你给我一个选题，我帮你从零写一篇公众号文章，自动匹配写作风格，生成大纲，渲染全文。你确认之后，一边把富文本注入剪贴板直接去微信粘贴，另一边生成 MDX 推到你的个人网站，git push 触发 Vercel 自动部署。

一条线，从想法到两边发布。

之前它是一个命令行工具——v2.1，你得记子命令、参数名。上周我把它重构成了一个 Agent。现在你对着终端说人话就行，不用学任何命令。今天来分享一下这个升级过程，以及我在 vibe coding 里怎么安排 AI 干活的。

第一段（1.5 分钟） — 项目本身在做什么

先说一下这个项目解决了什么问题。

我平时写公众号，也维护一个个人网站 khililo.xyz。以前发一篇文章的流程是这样的：写 Markdown → 转 HTML → 粘贴到微信后台调格式 → 再手动同步到网站仓库 → git push。每一步都在切换工具，而且格式总是不对——微信有自己的 HTML 规则，pulldown-cmark 转出来的 `<hr>` 标签微信直接忽略，中文编码在剪贴板里会被 Windows 的 UTF-16 截断。

所以 v2.0 的时候我先解决了一个硬问题：Markdown → 微信兼容 HTML → 跨平台剪贴板注入。WSL2 下用 PowerShell 的 MemoryStream 绕过 .NET 编码层，Linux 下走 xclip 或 wl-copy。这个链路跑通之后，你在终端里生成的文章，Ctrl+V 就能直接进微信后台。

然后是风格系统。你给一个 URL，Agent 用 Jina Reader 抓取正文，LLM 从十个维度逆向分析写作风格——标题策略、开头模式、段落节奏、句式特征、论证手法、情绪基调——生成一个可以直接当 system prompt 用的风格文件。之后你写文章说"用轻辩风格"，它会模糊匹配到对应的风格文件，把这个风格注入到全文渲染的 prompt 里。

核心架构是双通道 LLM：第一遍出大纲骨架，第二遍按骨架和风格渲染全文。这是为了解决单次长文生成容易逻辑坍缩的问题——先定结构再填内容，比让 LLM 一口气写 2000 字稳定得多。

第二段（1.5 分钟） — 怎么用 AI 做这个项目

讲讲我是怎么干活的。整个 v2.1 到 v3.1 的重构，我自己一行代码没写。

我用的是 Claude Code。工作方式是这样的：我把 ARCHITECTURE.md 写成详细的设计文档——模块职责、函数签名、状态机流转、测试要求——然后按依赖拓扑拆成 Wave。

Wave 1 是三个无依赖的模块：error.rs、intent.rs、website.rs。三个子 Agent 并行开发，各写各的，写完各自跑测试。Wave 1 全部通过之后 Wave 2 才启动——styles.rs、generate.rs、learn.rs、update.rs、io.rs 五个模块并行。Wave 3 是 repl.rs，串联所有模块。最后一波是 main.rs 退化启动分流器。

每波拆成 P7 agent——就是 Senior Engineer 级别的子任务。我作为主 Agent 负责审方案、派任务、看测试结果。Agent 写完代码后必须自审查三问：接口兼容吗？边界处理了吗？是 proper fix 还是 workaround？

（屏幕录制切到终端，展示一下 `cargo test --lib` 的 132 passed）

全程 14 个子 Agent，我自己做的就是设计、审方案、合并。这个工作方式让我觉得最有意思的点是：你的角色从“写代码的人”变成了“设计系统的人”。你的产出不是代码行数，是设计文档的质量。

第三段（1.5 分钟）— 升级的核心：从命令到对话

回到项目本身。v3.0 最大的架构变化就一个：交互模型翻了。

之前是 CLI——你想干什么得先学会它的命令。现在是一个 REPL 循环，通了一个 LLM 意图分类层。你的自然语言输入先经过一条三路径：

第一步，快速预检。“OK”、“算了”这种高频短词不走 LLM，0 成本直接返回。第二步，走 LLM 分类。一个极小的 prompt，大概 200 token，把你的输入分类成结构化 Intent。“帮我整一篇关于认知偏差的，犀利点”→ 正确识别成 Generate，topic 和 style 都提取出来。第三步，如果 LLM 挂了，回退到关键词匹配。

我一开始其实没做 LLM 分类。最早是纯关键词匹配——找“写一篇”、“用”、“风格”这些字眼。结果我自己试用时候说了句“帮我整一篇认知偏差的”，直接被拒。因为找不到关键词。这个错误很低级，但就是这个瞬间让我意识到 Agent 的核心不是规则，是理解。

然后是对话状态机。用户和 Agent 的交互有上下文——在 Idle 下说“OK”没意义，在 WaitingForPublish 下说“加个分论点”也没意义。我用了一个四状态的状态机：Idle → WaitingForFulltext → WaitingForPublish → 回到 Idle。不在状态的输入会被明确拒绝。

有一件事我后来砍掉了：大纲确认。一开始我设计的是三个检查点——确认大纲、确认全文、确认发布。上线之后发现第一个检查点纯粹是多余操作。LLM 大纲质量已经很稳定，再让用户点一次头就是增加摩擦。砍掉之后流程变成选题 → 大纲全文一气呵成 → 全文确认 → 发布确认。

第四段（1 分钟）— 双平台发布 + 会话持久化

发布链路值得一提。文章生成后两条轨同时走：

公众号这边——Markdown 转微信兼容 HTML，构建 CF_HTML 格式的二进制数据，PowerShell MemoryStream 注入系统剪贴板。你打开微信后台 Ctrl+V，富文本直接到位。因为我用的是个人订阅号，没有 API 权限，所以不做接口对接，剪贴板是最务实的方案。

网站这边——khililo.xyz 是 Next.js + next-mdx-remote，文章以 MDX 格式存在 content/signal/ 目录下。Agent 自动生成 frontmatter，写入文件，然后 git commit + push。Vercel 监听到 push 自动部署。你不

需要打开网站仓库。

v3.1 我加了一个东西：会话持久化。REPL 的 stdin 从阻塞读改成了 tokio 异步事件循环，每次 dispatch 后自动把整个 Repl 结构体序列化到 JSON 存到磁盘。进程死了、终端关了——下次启动检测到文件就提示恢复。这个场景不需要数据库，一个 JSON 文件够用。

第五段（1 分钟） — 数字和现状

看一眼数据。v2.1 的时候 main.rs 667 行，一个上帝函数。现在 126 行，14 个源文件，132 个单元测试。最大模块是 repl.rs，1116 行——编排层就是会胖。

技术栈：全程 Rust，LLM 走 DeepSeek 的 OpenAI 兼容端点，每次分类调用的成本大概万分之一美元。前端网站是 Next.js 16 + Tailwind 4。

说说不足。第一个，learn 和 update 子命令内部有 process::exit，在 REPL 里会直接杀进程，还没改成 Result。第二个，好几个模块里有重复的辅助函数副本——env_var、md_to_wechat_html 之类的，应该抽到一个 utils.rs 但还没做。第三个，clap 依赖已经没用了但懒得删。这些都在技术债列表里，不影响使用，但看着不舒服。

说这些不是为了显得谦虚。就是觉得做一个东西不需要完美才能拿出来。它能跑，能解决你的问题，剩下的慢慢收拾。

结尾（30 秒）

代码在 GitHub 开源，搜 cognitive-writer 就能找到。如果你也想试试用 AI Agent 帮自己写文章——或者你更感兴趣的是怎么安排多个 AI 并行干活的——可以看看 docs 目录下的 ARCHITECTURE.md 和 EVOLUTION.md，我把整个设计思路和演进过程都写在里面了。

就这样。